Python Programming

Features

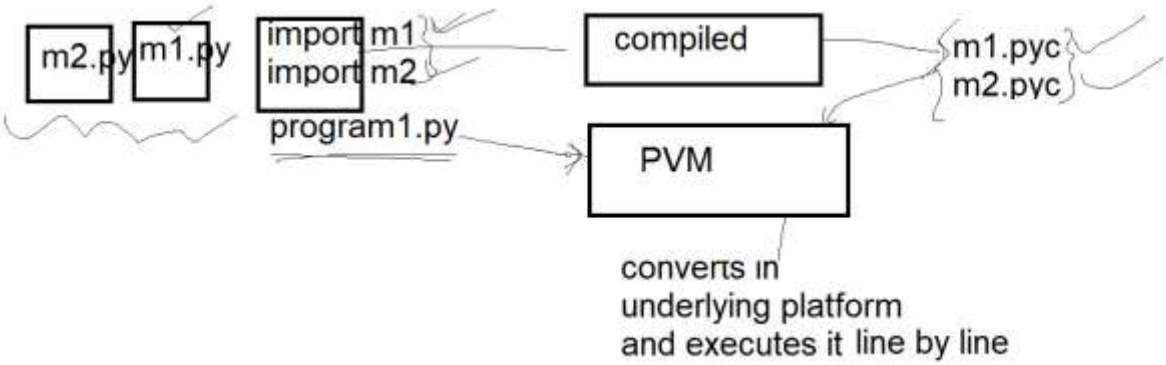
- Platform independent→If we write a program in one operating system, and if we can run it on any other operating system, without making any changes. If we write code on windows OS, and if I can run it on Linux as well as on mac machine then it is called as Platform independent.
- Machine independent-→ If the program is hardware independent, then it is called as machine independent.
- Easy to learn-→Its footprint is very small, no pointers are there, and the size of code is smaller as compared to other languages
- Easy to read the program-→ Indentation of the code is necessary, without indentation your code will not run
- Garbage collector is there--→Unused memory clearing will be done by garbage coolector, so there is no problem of memory leakage.
- It supports Object oriented programming.
- Network programming is also possible→ since in python you may write socket programming, so you may write programs like chatbots.
- It supports multithreading and multiprocessing.
- GUI programming is also possible-→ QtPy or tkinter that supports GUI programming
- Data base connectivity is also possible, you may connect to mysql, oracle or mongodb, Sqlite
- Modular programming is also possible.

Install python

1. Download python latest version from
   https://www.python.org/downloads/
2. Run the msi file, follow the step by step process to install it
3. It will be installed at following path
   C:\Users\anilk\AppData\Local\Programs\Python
4. To check python version
   Open cmd prompt and use following command
   Python –version
5. To start python Shell to test single command, use
   C:/system32>python
   >>>

Characteristics

1. Python is dynamically typed, and java is statically typed
   A=10 A=" Kishori"
2. Python is interpreter-→ it converts the code line by line and it executes line by line.
3. In python modules can be saved in compiled version



1. To execute program in cmd
   D:\pythondemos>python hello.py

2. To store compiled version, as byte code
   D:\pythondemos>python -m hello.py

   In python
   Basic data type are:-
   1. Number
   2. String

Operators are.

Arithmetic operators

+, -, *, /, //(integer division)

++ and - - is not there.

Logical operators

and , or, not

Relational operators

>, <, >=, <=, ==, !=

Bitwise operators

&, | , ~,>>, <<

Walrus operator(it is added in 3.8)

:=

n=10 If n>20:	If n=10:	If ((n:=10)>20)

In python functions are written by using def keyword

```
def f1():
    return 12
```

To a function you may use default parameters to make it optional parameter, c is optional parameter
A and b are mandatory parameters
	def f1(a,b,c=23):
	print(a,b,c)
	return 12
	#to call function
	f1(12,13)
	f1(12,13,14)
	#call using keyword parameters
	f1(c=34,a=20,b=46)

If a function call itself, then it is called as recursive function	def addition(n):
	if n==1:
	return 1
	else:
	print(f"in recurssiove call for {n-1}")
	return n+addition(n-1)
	
	ans=addition(6)
	print(ans)

Basic data types in python

Number-→ int, float,complex

String-→ string can be enclosed in ‘’(single quote),””(in double quote),””” “””(triple double quote), ‘’’ ‘’’(triple single quote)

Boolean-> True, False

Variables are of 2 types:

1. Mutable--- The value can be changed.
2. Immutable—The value cannot be changed. All basic data types are immutable.

Number functions

bin(num)	Prints binary representation of the number
int(num)	Converts the string or binary representation in decimal format

Round(num,precision)	This function will show precision number of digits after decimal point
math.sqrt(num)	It will show the sqrt of the number
Math.ceil(num)	It will remove the fraction part and gives the next number
Math.floor(num)	It will remove the fraction part and gives the same number

The functions that can be applied to numbers are available in math library

To import math library

import math

To see the list of functions in any module

dir(math)

To see the help of any one of the functions.

help(math.sqrt) -→ it will show the one liner quick help

String function

Internally the string is treated as list of characters and every character has 2 indexes +ve and -ve

Splicing of the string

T	H	I	S		I	S		S	T	R	I	N	G
0	1	2	3	4	5	6	7	8	9	10	11	12	13
-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

To find length of the string	len(s)	S="this is string" len(s)=14
To find the last caharectes	S[-1]	G
To find second last character	S[-2]	N
To find values from 3 rd index to 1oth	S[3:11]	s is str
To find values from the 5 till end	S[5:]	Is string
To find the string from th beginning till 5 th index	S[:6]	This I
To find all characters at even index position	S[::2]	TI SSRN
To find all characters at odd index position	S[1::2]	TI SSRN
To print string in reverse order	S[::-1]	GNIRTS SI SIHT
TO PRINT FROM -9 TO 11	S[-9:11]	IS STR

s.upper()	Converts the string in uppercase
s.lower()	Converts the string in lowercase
s.startswith(substr)	Return true ,if the given string starts with substr
s.endswith(substr)	Returns true if the string ends with substr
s.find(substr,[start,end])	Return the position of 1 st occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, it returns -1 if substr not found
s.rfind(substr,[start,end])	Return the position of last occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, it return -1 if substr not found
s.index(substr,[start,end])	Return the position of 1 st occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, but it throws exception if the substr not found
s.rindex(substr,[start,end])	Return the position of last occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, but it throws exception if the substr not found
s.strip(string)	It will delete all occurrences of the characters from leftmost and rightmost side of the string
s.lstrip(string)	It will delete all occurrences of the characters from leftmost side of the string
s.rstrip(string)	It will delete all occurrences of the characters from rightmost side of the string
s.split(delimiter)	It will brek the string into parts at delimiter character and returns a list
“delimiter”.join(lst)	It will join all the strings from the list separated by delimiter

s.upper()	Converts the string in uppercase
s.lower()	Converts the string in lowercase
s.startswith(substr)	Return true ,if the given string starts with substr
s.endswith(substr)	Returns true if the string ends with substr
s.find(substr,[start,end])	Return the position of 1 st occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, it returns -1 if substr not found
s.rfind(substr,[start,end])	Return the position of last occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, it return -1 if substr not found
s.index(substr,[start,end])	Return the position of 1 st occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, but it throws exception if the substr not found
s.rindex(substr,[start,end])	Return the position of last occurrence of substr , if the start and end is not given then it searches the full string, otherwise it searches in the given range, but it throws exception if the substr not found
s.strip(string)	It will delete all occurrences of the characters from leftmost and rightmost side of the string
s.lstrip(string)	It will delete all occurrences of the characters from leftmost side of the string
s.rstrip(string)	It will delete all occurrences of the characters from rightmost side of the string
s.split(delimiter)	It will brek the string into parts at delimiter character and returns a list
“delimiter”.join(lst)	It will join all the strings from the list separated by delimiter
S.replace(oldsubstr,newsubstr,[count])	It will replace all occurrences of oldsubstr with newsubstr, if you specify count, then it will replace only first count number of occurrences
s.isalpha()	Returns true, if all the characters in the string are alphabets
s.isalnum()	Returns true if the string has all characters as either alphabet or digits, no special characters are allowed
s.count(substr)	It will show, the number of occurrences of the given string

String Type	Example	Python `.isdecimal()`	Python `.isdigit()`	Python `.isnumeric()`
Base 10 Numbers	`'0123'`	True	True	True
Fractions and Superscripts	`'⅔', '2²'`	False	True	True
Roman Numerals	`'Ⅾ'`	False	False	True

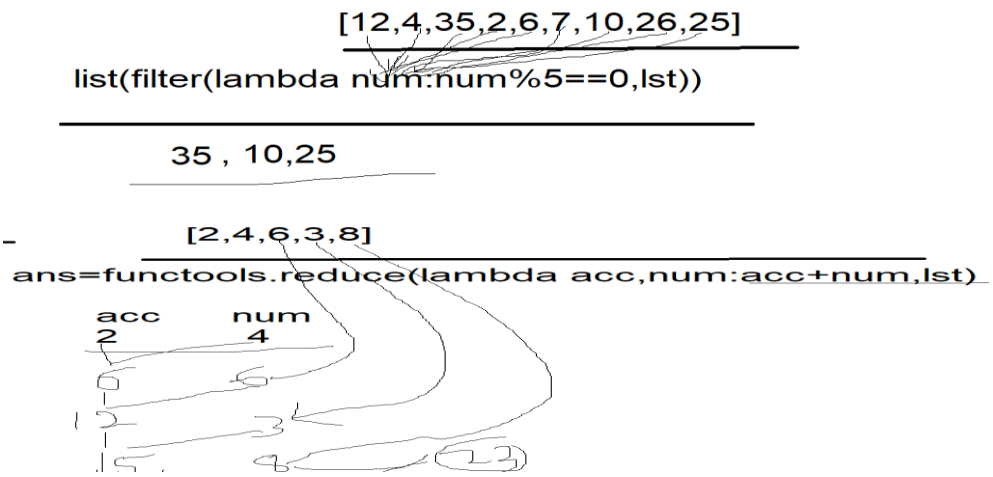
Other data types

1. Lists
    a. Lists allows you to store heterogeneous data.
    b. It is mutable data structure.
    c. Duplicate values are allowed.
    d. It is represented using []
    e. It is ordered collection, means the data will be stored in same sequence, as it is entered.
    f. Since it is ordered, using index is possible, hence data can be retrieved randomly.

lst.index(value)	Returns the position of first occurrence of the given value
Lst.append(value)	To add one value at the end of the list, and increase the length of the list by 1
Lst.extend(iterable)	To add multiple values from the given iterable at the end of the list, and increases the length by len(iterable)
Lst.insert(pos,value)	To add the value at the given position, if position is beyond the length of the list, then it will add at the end
Lst.pop([num])	To delete the last value from the list, if num is not given, but if we specify the num, then it will delete it from the given position, if the num is within limits of the length of the list, or it should not be empty
Lst.remove(value)	To delete the first occurrence of the given value from the list
Lst.count(value)	Display number of occurrences of the given value
Lst.clear()	To delete all the values from the list
Lst.copy()	To create a shallow copy of the list
Lst.reverse()	To reverse the list contents, it will change the original list
Lst.sort()	To arrange the data in the sorted order, by default it will arrange the data in ascending order,

	To arrange it in descending order use lst.sort(reverse=True) But sorting is possible only on the list which is homogeneous

filter(function,iterable)	Will internally generate the loop to navigate through the list one by one, and apply the function, if function returns True, then keep the value, else remove it Original iterable will remain as it is
map(function,iterable)	Will internally generate the loop to navigate through the list one by one, and apply the function, return the answer of the function, one for each value in the list
Reduce(function,iterable)	Will internally generate the loop to navigate through the list one by one, and apply the function, return the answer single value for all values from the list



2. Tuple
    a. It is immutable.
    b. It is represented using ().
    c. Mainly tuples are used to return multiple values from a function.
    d. Ordered collection, so we can access data by using index
    e. Since the indexing is possible, access to the data is random
    f. Duplicate values are allowed.
    g. Tuples are also iterable.
3. Set
4. Frozenset
5. dictionary

1. Tuple
    a. It is immutable.
    b. It is represented using ().
    c. Mainly tuples are used to return multiple values from a function.
    d. Ordered collection, so we can access data by using index
    e. Since the indexing is possible, access to the data is random
    f. Duplicate values are allowed.
    g. Tuples are also iterable.

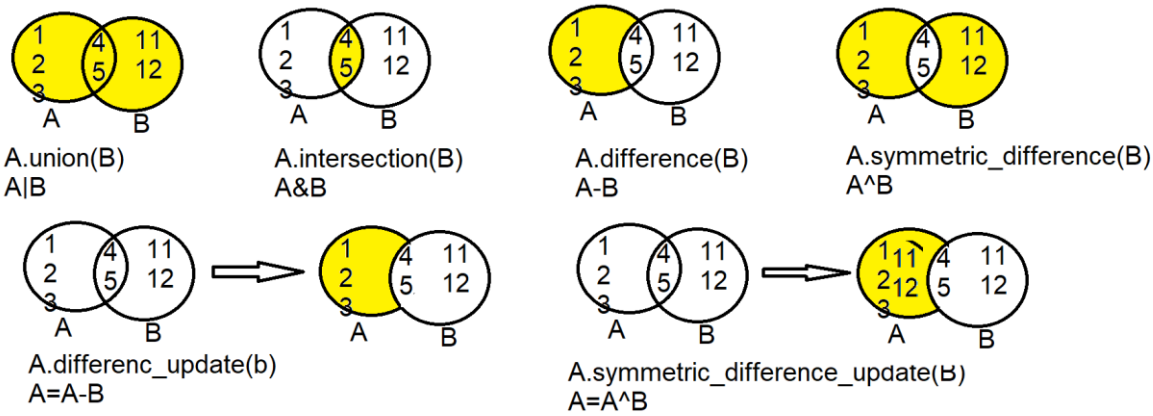
Uses of tuple

a) To return multiple values from the function'
b) To pass variable number of arguments to a function
c) To make the data immutable, so that no one can change it further

2. Set
    a. It allows to store unique values, and stores only immutable values.
    b. It is mutable.
    c. It is represented by using {}
    d. It is unordered list; hence indexing is not possible, and hence data cannot be read randomly.

s.add(value)	It add a value in the set if it does not exists in the set
s.update(iterable)	It will add multiple values from the iterable one by one in the set
s.pop()	It will randomly delete the value and return it.
s.remove(value)	It will delete the given value if exists, else throw exception
s.discard()	It will delete the given value if exists, else it will ignore
s.copy()	To create a shallow copy of the set. So that changes in the original set will not be reflected in the copied set
S.clear()	To delete all the values from the set
s.union(s1) s|s1	To get all values from s as well as s1
s.intersection(s1) s&s1	To get only values which are common
s.difference(s1) s-s1	To get only values which are in s and not in s1
s.symmetric_difference(s1) s^s1	To get only values in either s or in s1
s.difference_update(s1)	To get only values which are in s and not in s1, but it will modify s, by overwriting it with the result

s. symmetric_difference_update(s1)	To get only values in either s or in s1, and also overwrite the s with the result



3. Frozenset
    a. These are sets which are immutable
4. dictionary

To download Anaconda and install it

https://www.anaconda.com/download

Dictionary:

1. It is mutable.
2. It stores key, value pair. Keys are unique. Any immutable object can be the key
3. It is ordered collection.
4. The value can be retrieved very fast, if you know the key. But if you want to find the key based on value, then you need to navigate through the dictionary.
5. Dictionary is represented by using {}
6. To create a empty dictionary you can use, d={} , to create empty set s=set()

To add a key

C={}

c['DAC']=240 # a new key -value pair will be added

c['DAC']=300 #if key exists it will overwrite the value

D1.update(d2)	Add all keys of d2 in d1, if any of the key is common, then d1's keys value will be overwritten by d2's keys value D1={'a':10,'b':20} d2={'b':300,'c':30} d1.update(d2) will change d1 ={'a':10,'b':300,'c':30}
D1.fromkeys(lst,default value)	create a new dictionary by adding every value from list as key value of all keys will be None, if the default value is not given
d.pop(key,[default value)	It will delete the pair of key-value if the given key exists and returns the value. But if key does not exist: And default value is given, then it will return default value, otherwise it will throw andexception
d.popitem()	It will delete the last key-value pair, it will return a tuple
d.copy()	It will create a shallow copy of the dictionary
d.clear()	It will delete all key-value pairs from the the dictionary
d.get(key,[default value])	It will return the value of the given key, if key exists, otherwise, if key does not exists, and if default value is not given, then it will return None, if deafault value is given, it will return default value
d.setdefault(key,[default value])	It will return the value of the given key, if key exists, otherwise, if key does not exist, and it will add a new key-value pair in the dictionary and returns default value.
d.Keys()	It will return all keys
d.values()	It will return all values
d.items()	It will return a iterable, each value will be a tuple of (key,value)

Regular expression

\d	One single digit

\D	One non digit character
\b	Boundary character
\B	Nonboundry characters
\w	One word character[0-9A-Za-z_]
\W	Non word charcaters
\s	One space character
\S	One nonspace character

	or	^or	Or$	\bor\b	\bor	
There is tailor	Y	N	Y	N	n	
There is a origami class	Y	N	N	N	Y	
This or that	Y	N	N	Y	Y	
Origami is good	y	Y	N	N	Y	
This is normal	y	n	n	n	n	

S=This is string

^\w+\s\w+\s\w+$---matches all the lines which has exact 3 words separated by space

^\w+\s\w+\s\w+-→ matches all the line which has atleast 3 words

\w+\s\w+\s\w+$-→ matches with lines ending with 3 words, before that anything can be there

S="something is there somewhere"

s.*e

S="something is there somewhere"

s.*?e

import re

re.search(pattern ,string, ,flags)	It will give the MatchObject with information about first occurrence of the string that matches with the pattern, the pattern can match anywhere in the string
re.match(pattern ,string, flags)	It will give the MatchObject with information about first occurrence of the string that matches with the pattern, the pattern can match only at the beginning of the string, even if ^ is not there still the pattern will be matched at the beginning

Re.findall(pattern ,string, ,flags)	It will return a list of string, where the pattern matches
re.finditer(pattern,string, flags)	It will return a list of matchObjects, where the pattern matches
Re.sub(pattern,string,newstring,flags)	It will replace all occurrences of the given pattern with the new string
Re.compile(pattern,flags)	Returns the regular expression object